

Estructura de datos

Estructuras lineales estáticas

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

- 1 Introducción a las estructuras de datos
- 2 Arreglos
- 3 Declaración y acceso
- 4 Inserción
- 5 Eliminación
- 6 Búsqueda
- 7 Ciclo `for-each`
- 8 Matrices
- 9 Recorrido por filas
- 10 Recorrido por columnas
- 11 Matrices Irregulares
- 12 Recorrido con `for-each`
- 13 Aplicaciones



¿Qué es una estructura de datos?

Definición

Una **estructura de datos** es una **forma organizada** de **almacenar, representar y manipular datos** para **resolver problemas computacionales** de manera eficiente.

- Permite **organizar** información en **memoria**.
- **Facilita operaciones** como **insertar, eliminar, buscar y recorrer** datos.
- Influye directamente en la **eficiencia** de los algoritmos.
- Su elección **depende del problema** que se desea resolver.

Clasificación de las estructuras de datos

Se clasifican por...

– **ORGANIZACIÓN LÓGICA**

– **LINEALES**

- Arreglos
- Listas
- Pilas
- Colas

– **NO LINEALES**

- Árboles
- Gráfos

– **USO DE MEMORIA**

- **Estáticas:** tamaño fijo
- **Dinámicas:** aumentan o disminuyen

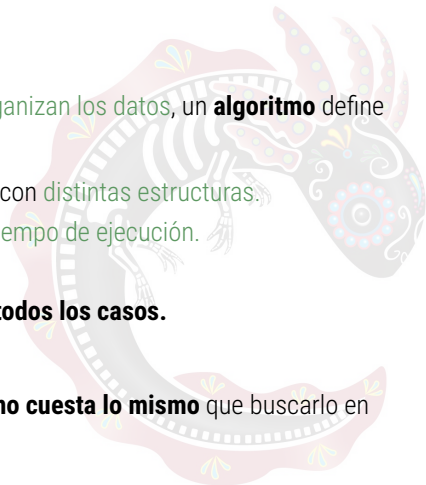
Estructuras de datos y algoritmos

Una **estructura de datos** define cómo se organizan los datos, un **algoritmo** define cómo se procesan.

- Un mismo problema puede resolverse con distintas estructuras.
- La estructura seleccionada afecta el tiempo de ejecución.
- También afecta el uso de memoria.
- No existe una estructura “mejor” para todos los casos.

Ejemplo

Buscar un dato en un arreglo desordenado no cuesta lo mismo que buscarlo en un arreglo ordenado.



Arreglos unidimensionales

Definición

Un **arreglo** es una **estructura de datos** que permite **almacenar** una **colección de elementos del mismo tipo**, identificados mediante un **índice**.

- Constituye una **estructura lineal**.
- Los elementos **mantienen** un **orden definido**.
- En **Java** ☞, los **índices** comienzan en **0**.
- Su **tamaño** se **establece** durante su **creación**.
- Permite **acceder directamente** a **cualquier elemento** mediante su **índice**.

Ejemplo

Un arreglo de **tamaño 5** utiliza los índices:

0, 1, 2, 3, 4

Representación de un arreglo

datos

15	28	10	42	7
0	1	2	3	4

- Cada elemento ocupa una **posición** dentro del arreglo.
- La **posición** se identifica mediante un **índice**.
- Para un arreglo de n elementos, los **índices válidos** son:

$$0 \leq i < n$$

con $i, n \in \mathbb{Z}$

Declaración y creación de arreglos

Declaración

```
int[] datos;
```

La declaración crea una **variable de referencia**, pero todavía **no crea** el arreglo.

Creación

```
datos = new int[5];
```

Declaración y creación

```
int[] datos = new int[5];
```

¡Importante!

En **Java** ☕, los arreglos son **objetos**; por ello se crean utilizando el operador **new**.

Inicialización de arreglos

Java  permite **inicializar un arreglo** durante su **declaración**:

```
int[] datos = {15, 28, 10, 42, 7};
```

También es posible **asignar los valores individualmente**:

```
int[] datos = new int[5];
```

```
datos[0] = 15;
```

```
datos[1] = 28;
```

```
datos[2] = 10;
```

```
datos[3] = 42;
```

```
datos[4] = 7;
```

Tamaño

La **propiedad o atributo** `length` permite conocer la **capacidad** del arreglo:

```
System.out.println(datos.length);
```

Acceso a los elementos

Para **acceder a un elemento** se utiliza su **índice**:

```
int[] datos = {15, 28, 10, 42, 7};  
int valor = datos[2];  
System.out.println(valor);
```

El resultado será:

10

Visualmente:

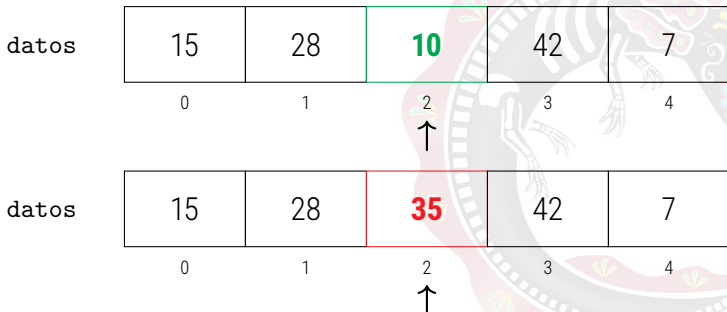
datos	15	28	10	42	7
	0	1	2	3	4

↑

Modificación de elementos

Para **modificar** un elemento:

```
datos[2] = 35;
```



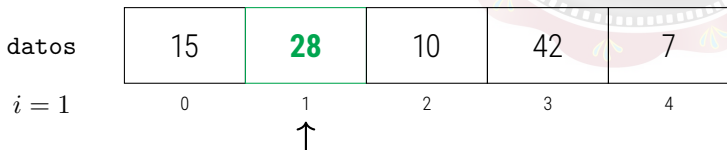
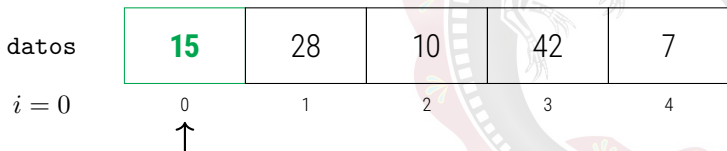
¡Importante!

Acceder a una posición **fuera de los límites** del arreglo genera la excepción `ArrayIndexOutOfBoundsException`.

Recorrido de un arreglo

Una **operación fundamental** consiste en **visitar todos sus elementos**.

```
int[] datos = {15, 28, 10, 42, 7};  
  
for (int i = 0; i < datos.length; i++) {  
    System.out.println(datos[i]);  
}
```



Recorrido de un arreglo

datos

15	28	10	42	7
----	----	----	----	---

 $i = 2$

0

1

2

3

4



datos

15	28	10	42	7
----	----	----	----	---

 $i = 3$

0

1

2

3

4



datos

15	28	10	42	7
----	----	----	----	---

 $i = 4$

0

1

2

3

4



Recorrido de un arreglo

Para un arreglo de n elementos se realizan las **siguientes operaciones**:

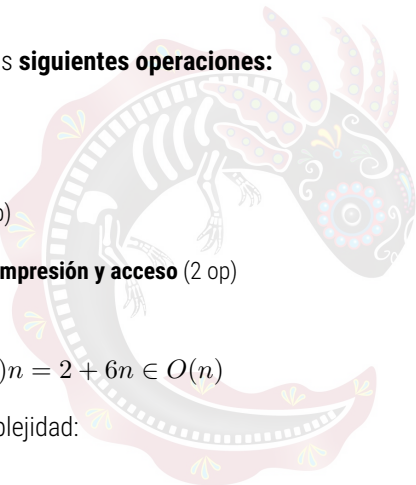
- Una **asignación inicial** (1 op)
- Una **comparación final** (1 op)
- Por **cada iteración** realizamos:
 - Una **comparación** y una **consulta** (2 op)
 - Un **incremento** (2 op)
 - El bloque contiene **dos instrucciones**: **impresión y acceso** (2 op)

Si planteamos una ecuación tenemos

$$T(n) = 1 + 1 + (2 + 2 + 2)n = 2 + 6n \in O(n)$$

por lo que se dice que el recorrido tiene complejidad:

$$O(n)$$



Inserción lógica

Un arreglo tiene **tamaño fijo**; por tanto, **insertar un elemento** no significa **aumentar físicamente** su tamaño.

Podemos utilizar una **variable** para indicar **cuántas posiciones** están ocupadas:

```
int[] datos = new int[10];  
int size = 4;
```

```
datos[0] = 10;  
datos[1] = 20;  
datos[2] = 30;  
datos[3] = 40;
```

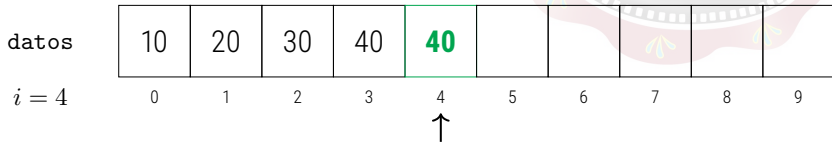
datos	10	20	30	40						
size= 4	0	1	2	3	4	5	6	7	8	9

Aunque `datos.length= 10` solamente existen 4 **elementos** almacenados.

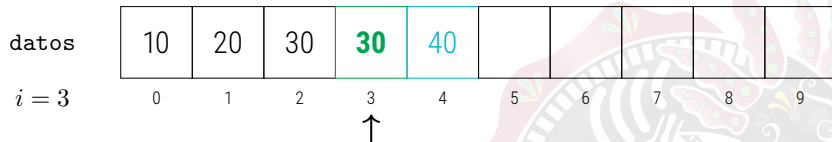
Inserción en una posición

Para insertar un elemento en una **posición intermedia** es necesario **desplazar** los **elementos posteriores**.

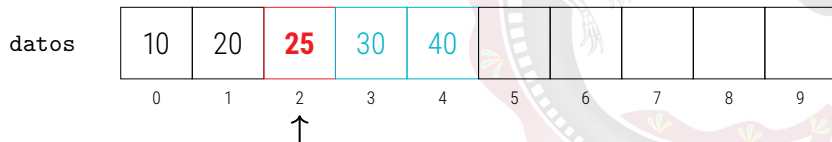
```
int index = 2;  
int value = 25;  
  
for (int i = size; i > index; i--) {  
    datos[i] = datos[i - 1];  
}  
  
datos[index] = value;  
size++;
```



Inserción en una posición



El ciclo termina



Complejidad

En el **peor caso**, la inserción requiere desplazar n elementos:

$$O(n)$$

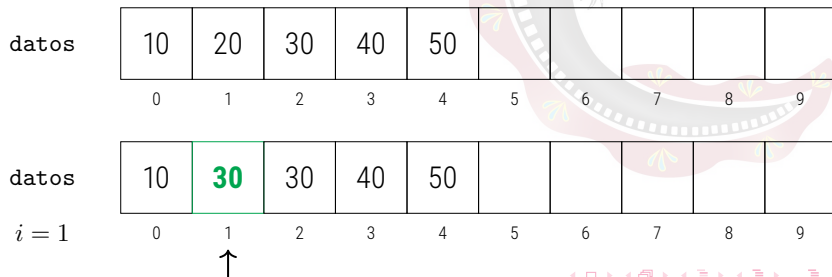
Eliminación lógica

Para eliminar un elemento es necesario **desplazar hacia la izquierda** los elementos posteriores.

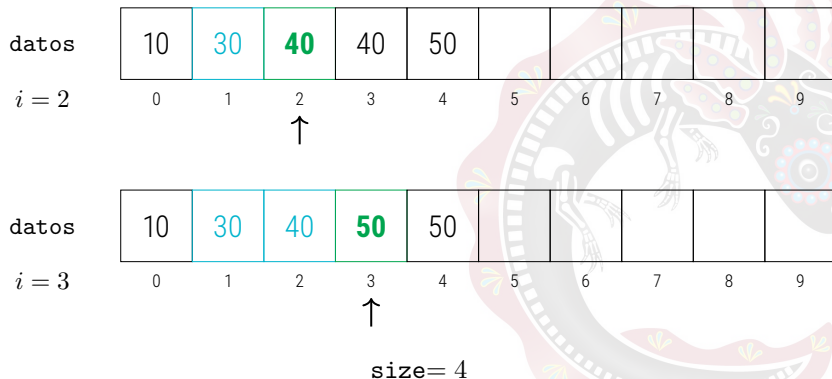
```
int index = 1;

for (int i = index; i < size - 1; i++) {
    datos[i] = datos[i + 1];
}

size--;
```



Eliminación lógica



Complejidad

En el **peor caso**:

$$O(n)$$

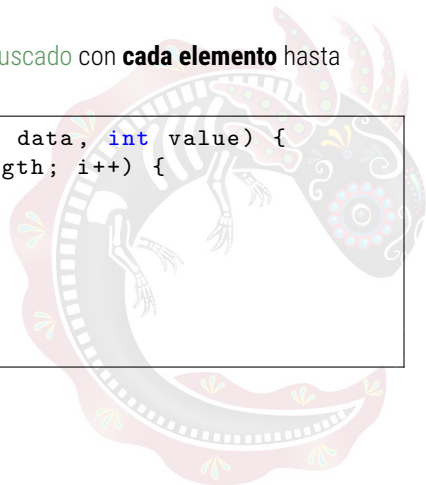
Búsqueda secuencial

La **búsqueda secuencial** compara el **valor buscado** con **cada elemento** hasta encontrarlo o llegar al final.

```
public static int search(int[] data, int value) {  
    for (int i = 0; i < data.length; i++) {  
        if (data[i] == value) {  
            return i;  
        }  
    }  
    return -1;  
}
```

Complejidad

- Mejor caso: $O(1)$.
- Peor caso: $O(n)$.



Búsqueda binaria

La **búsqueda binaria** divide repetidamente el **espacio de búsqueda** en **dos partes**.

¡Importante!

El arreglo debe encontrarse **ordenado**.

Por ejemplo:

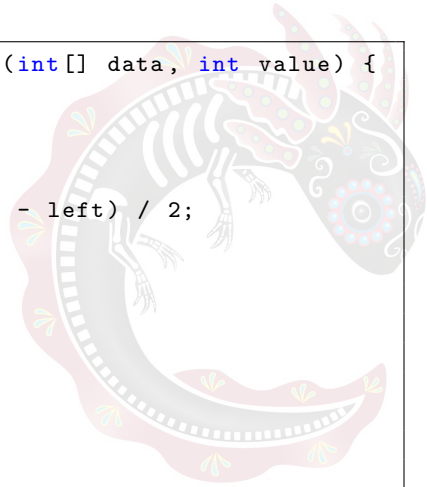
[5, 12, 18, 25, 31, 42, 57, 63, 78]

Para buscar 57:

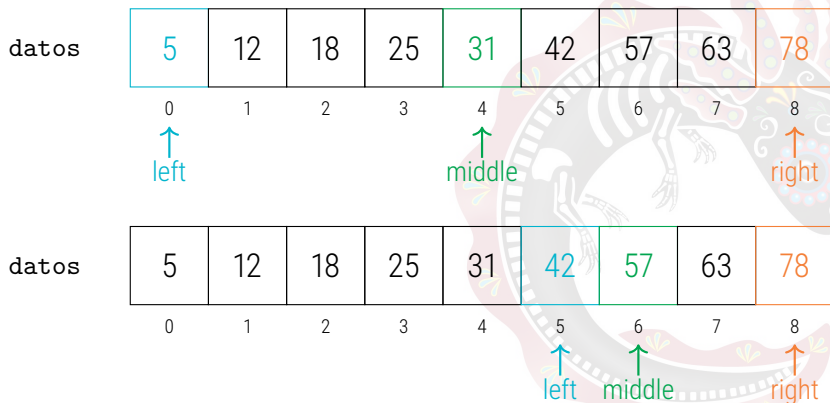
- 1 Se examina el elemento central.
- 2 Se determina en qué mitad puede encontrarse el valor.
- 3 Se descarta la otra mitad.
- 4 El procedimiento se repite.

Implementación de la búsqueda binaria

```
public static int binarySearch(int[] data, int value) {  
    int left = 0;  
    int right = data.length - 1;  
  
    while (left <= right) {  
        int middle = left + (right - left) / 2;  
  
        if (data[middle] == value)  
            return middle;  
        if (data[middle] < value)  
            left = middle + 1;  
        else  
            right = middle - 1;  
    }  
    return -1;  
}
```



Ejemplo de búsqueda binaria



Complejidad

- Mejor caso: $O(1)$.
- Peor caso: $O(\log n)$.

Búsqueda secuencial vs. binaria

Característica	Secuencial	Binaria
Requiere datos ordenados	No	Sí
Mejor caso	$O(1)$	$O(1)$
Peor caso	$O(n)$	$O(\log n)$
Implementación	Simple	Mayor

¡Importante!

Una **búsqueda binaria** es **más eficiente** sobre **grandes conjuntos de datos**, pero mantener los **datos ordenados** también puede tener un **costo**.

Arreglos de objetos

Los **arreglos** también pueden almacenar **referencias a objetos**.

```
Estudiante[] grupo = new Estudiante[3];  
  
grupo[0] = new Estudiante("Ana", 9.5);  
grupo[1] = new Estudiante("Luis", 8.7);  
grupo[2] = new Estudiante("Maria", 9.1);
```

Representación

El arreglo almacena **referencias**, los objetos se encuentran **almacenados** en el **heap**.

¡Importante!

Al crear el arreglo, sus posiciones **contienen inicialmente null**, los objetos **deben crearse por separado**.

Ciclo `for-each`

Definición

El ciclo `for-each` permite **recorrer secuencialmente** todos los **elementos** de un **arreglo** o colección, **sin utilizar** explícitamente un **índice**.

Sintaxis

```
for (tipo elemento : arreglo) { ... }
```

¡Importante!

El ciclo `for-each` es **apropiado** cuando necesitamos **recorrer todos los elementos** y **no necesitamos conocer o modificar** directamente su **posición** mediante un índice.

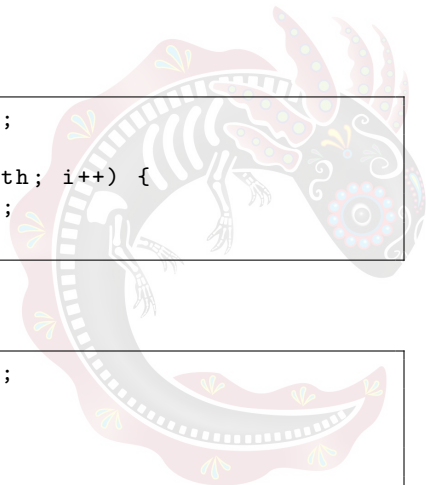
Comparación

Ciclo for tradicional

```
int[] datos = {10, 20, 30, 40};  
  
for (int i = 0; i < datos.length; i++) {  
    System.out.println(datos[i]);  
}
```

Ciclo for-each

```
int[] datos = {10, 20, 30, 40};  
  
for (int valor : datos) {  
    System.out.println(valor);  
}
```



Limitaciones del `for-each`

La **variable utilizada** por el `for-each` recibe el valor de **cada elemento** durante el recorrido, por lo que **el arreglo solo se consulta**.

¡Importante!

```
int[] datos = {10, 20, 30};  
for (int valor : datos) {  
    valor = valor * 2;  
}
```

Los elementos del arreglo **no se modifican**, para ello **es necesario acceder** a ellos mediante su **índice**.

Matrices

Un **arreglo bidimensional** puede utilizarse para representar una **matriz organizada** mediante **filas** y **columnas**.

```
int [][] matriz = {  
    {10, 20, 30},  
    {40, 50, 60},  
    {70, 80, 90}  
};
```

El acceso utiliza **dos índices**:

`matriz[filas][columna]`

Por ejemplo:

```
int valor = matriz[1][2];
```

produce:

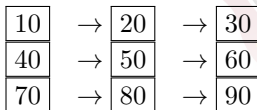
`valor = 60`

Recorrido por filas

Para **recorrer una matriz** se utilizan normalmente **ciclos anidados**.

```
for (int i = 0; i < matriz.length; i++) {  
    for (int j = 0; j < matriz[i].length; j++) {  
        System.out.print(matriz[i][j] + " ");  
    }  
    System.out.println();  
}
```

El **orden de recorrido** será:



Ejemplo: recorrido por filas (0,0)

$j = 0$

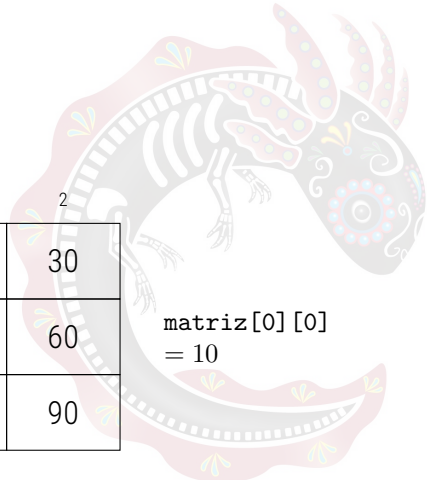
↓

0 1 2

$i = 0$ → 0

0	10	20	30
1	40	50	60
2	70	80	90

`matriz[0][0] = 10`



Ejemplo: recorrido por filas (0,1)

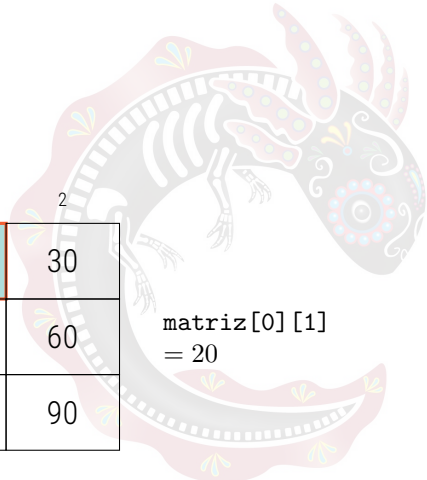


Diagram illustrating a row-major traversal of a 3x3 matrix. The current position is at row $i = 0$ and column $j = 1$, which contains the value 20. The matrix is shown with indices i and j indicating the current position.

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

`matriz[0][1] = 20`

Ejemplo: recorrido por filas (0,2)

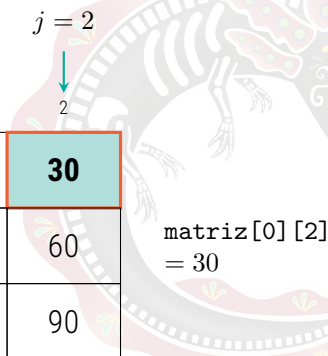


Diagram illustrating a row-major traversal of a 3x3 matrix. The current position is at row $i = 0$ and column $j = 2$, which contains the value 30. The matrix is shown with columns indexed 0, 1, and 2, and rows indexed 0, 1, and 2. The element at $(0, 2)$ is highlighted with a red border. The text `matriz[0][2] = 30` is displayed next to the matrix.

	0	1	2	
$i = 0 \rightarrow$	0	10	20	30
1	40	50	60	
2	70	80	90	

`matriz[0][2] = 30`

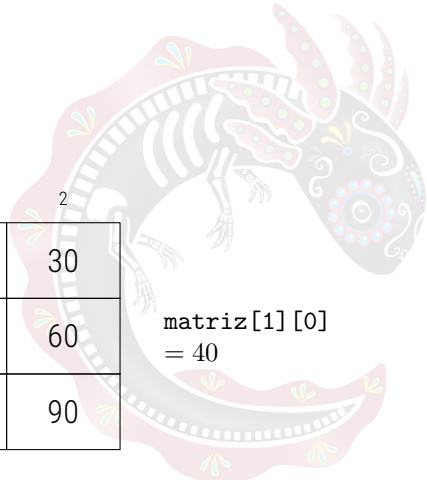
Ejemplo: recorrido por filas (1,0)

$j = 0$

$i = 1 \rightarrow$

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

`matriz[1][0] = 40`



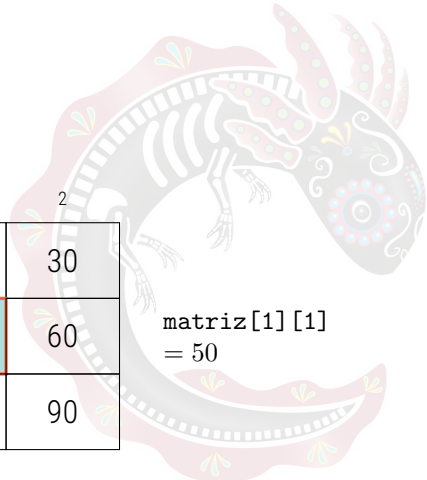
Ejemplo: recorrido por filas (1,1)

$j = 1$

$i = 1 \rightarrow$

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

`matriz[1][1] = 50`



Ejemplo: recorrido por filas (1,2)

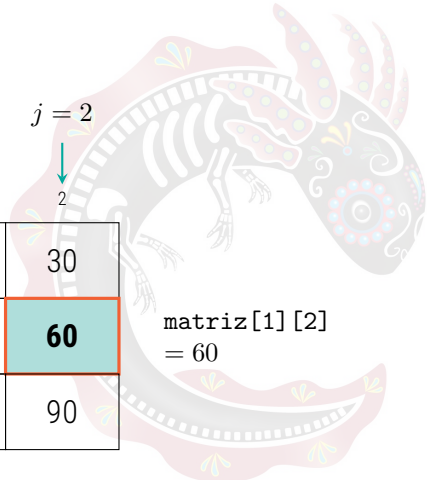


Diagram illustrating a row-major traversal of a 3x3 matrix. The current row is $i = 1$ and the current column is $j = 2$. The element at `matriz[1][2]` is 60.

	0	1	2
0	10	20	30
$i = 1 \rightarrow$ 1	40	50	60
2	70	80	90

`matriz[1][2] = 60`

Ejemplo: recorrido por filas (2,0)

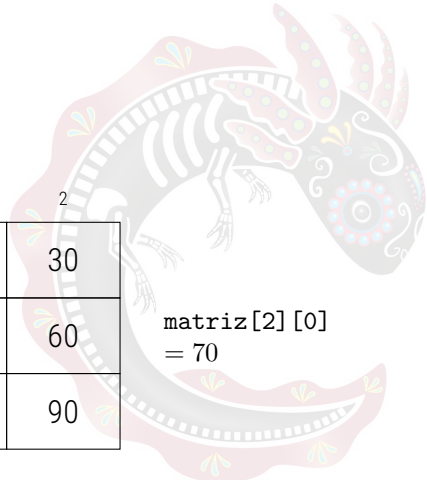
$j = 0$

↓

0 1 2

0	10	20	30
1	40	50	60
$i = 2$ → 2	70	80	90

`matriz[2][0] = 70`



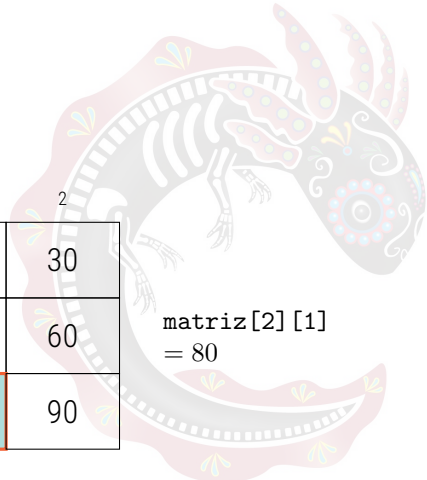
Ejemplo: recorrido por filas (2,1)

$j = 1$

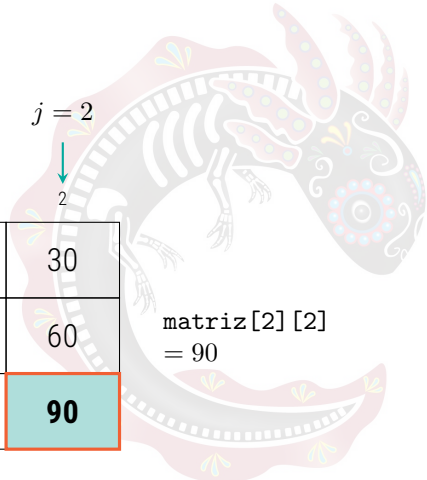
0 1 2

0	10	20	30
1	40	50	60
$i = 2$ → 2	70	80	90

`matriz[2][1]`
`= 80`



Ejemplo: recorrido por filas (2,2)



A 3x3 matrix illustrating row-major traversal. The current element being accessed is at row $i=2$ and column $j=2$, which contains the value 90. The matrix is shown with row and column indices. The value 90 is highlighted with a red border. To the right of the matrix, the code `matriz[2][2] = 90` is displayed.

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

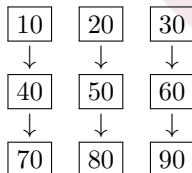
`matriz[2][2] = 90`

Recorrido por columnas

También es posible recorrer **primero las columnas**:

```
for (int j = 0; j < matriz[0].length; j++) {  
    for (int i = 0; i < matriz.length; i++) {  
        System.out.print(matriz[i][j] + " ");  
    }  
    System.out.println();  
}
```

El orden será:



Ejemplo: recorrido por columnas (0,0)

$j = 0$

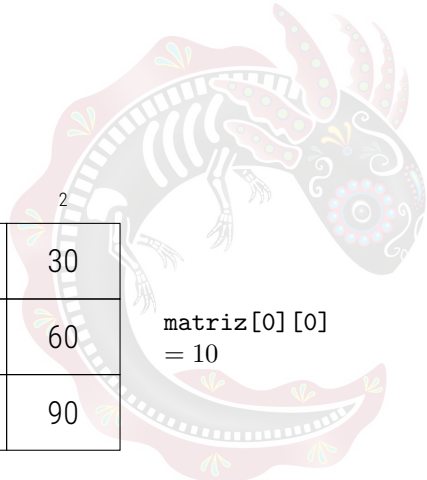
↓

0 1 2

$i = 0 \rightarrow$ 0

0	10	20	30
1	40	50	60
2	70	80	90

`matriz[0][0] = 10`



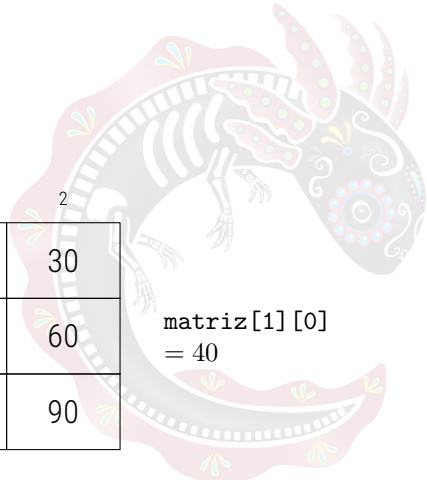
Ejemplo: recorrido por columnas (1,0)

$j = 0$

$i = 1 \rightarrow$

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

`matriz[1][0] = 40`



Ejemplo: recorrido por columnas (2,0)

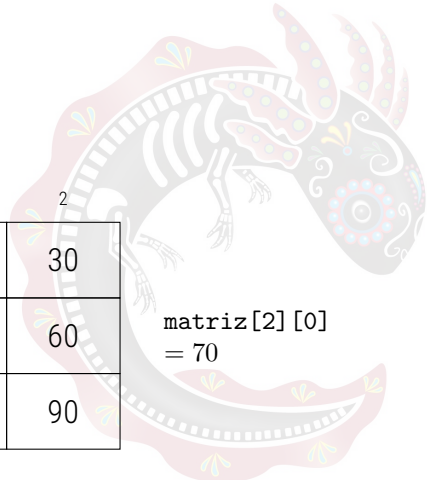
$j = 0$

↓

0 1 2

0	10	20	30
1	40	50	60
$i = 2$ → 2	70	80	90

`matriz[2][0] = 70`



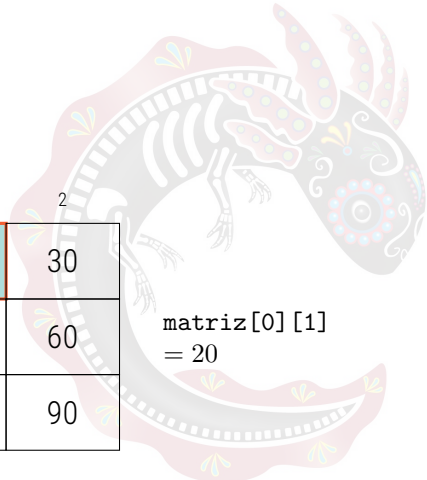
Ejemplo: recorrido por columnas (0,1)

$j = 1$

$i = 0 \rightarrow$

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

`matriz[0][1] = 20`



Ejemplo: recorrido por columnas (1,1)

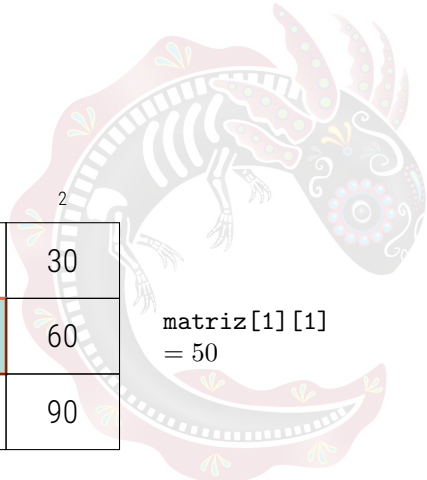


Diagram illustrating a column-major traversal of a 3x3 matrix. The matrix is shown with indices i (rows) and j (columns). The current element being accessed is at $i=1$ and $j=1$, which contains the value 50. The traversal is indicated by the label "Recorrido por columnas (1,1)".

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

$i = 1 \rightarrow$

$j = 1$

`matriz[1][1] = 50`

Ejemplo: recorrido por columnas (2,1)

$j = 1$

0 1 2

0 10 20 30

1 40 50 60

$i = 2$ → 2 70 **80** 90

`matriz[2][1] = 80`

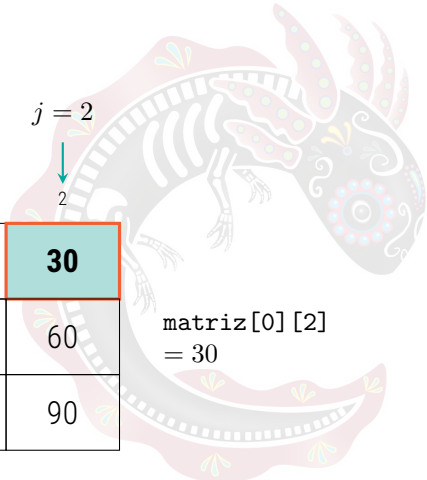
Ejemplo: recorrido por columnas (0,2)

$i = 0 \rightarrow$

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

$j = 2$

`matriz[0][2] = 30`



Ejemplo: recorrido por columnas (1,2)

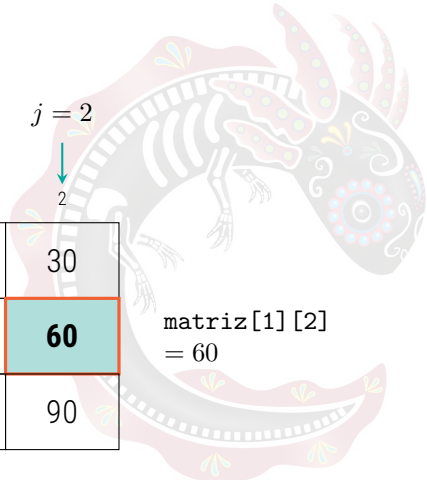


Diagram illustrating a column-major traversal of a 3x3 matrix. The matrix is shown with rows indexed 0, 1, 2 and columns indexed 0, 1, 2. The current element being accessed is at row $i = 1$ and column $j = 2$, which contains the value 60. The access is highlighted by a red border around the cell and an orange arrow pointing to the row index $i = 1$. A blue arrow points to the column index $j = 2$. The code snippet `matriz[1][2] = 60` is displayed next to the cell.

	0	1	2
0	10	20	30
$i = 1 \rightarrow$ 1	40	50	60
2	70	80	90

`matriz[1][2] = 60`

Ejemplo: recorrido por columnas (2,2)

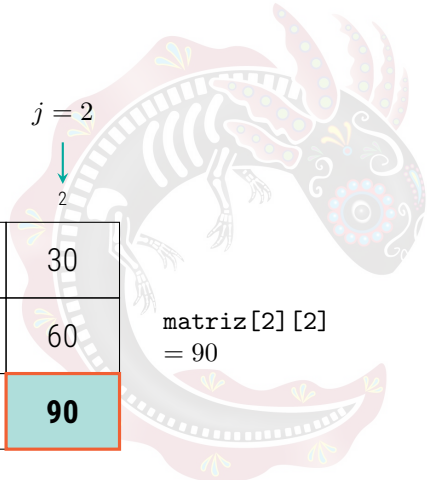


Diagram illustrating a column-major traversal of a 3x3 matrix. The matrix is shown with rows indexed 0 to 2 and columns indexed 0 to 2. The current position is at row $i = 2$ and column $j = 2$, which contains the value 90. The traversal is indicated by an orange arrow pointing to the row index $i = 2$ and a blue arrow pointing to the column index $j = 2$. The value 90 is highlighted in a teal box.

	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

$i = 2 \rightarrow$

$j = 2$

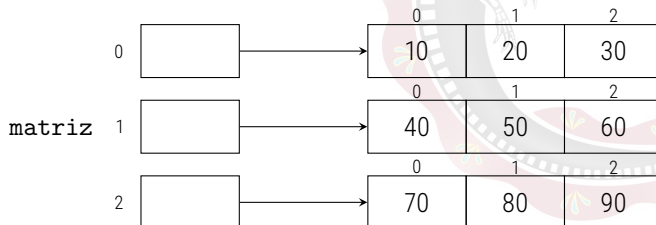
`matriz[2][2] = 90`

Representación de matrices en Java

¡Importante!

En **Java** ☕, un **arreglo bidimensional** no es necesariamente **una matriz rectangular** almacenada como un **único bloque**.

Un **arreglo bidimensional** realmente es un **arreglo de arreglos**



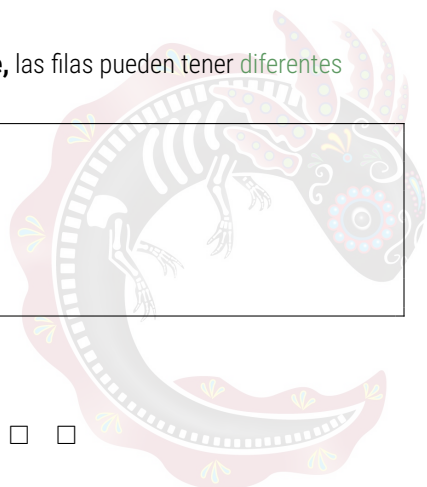
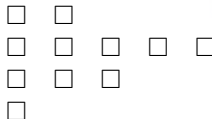
Matrices irregulares

Como cada fila es un **arreglo independiente**, las filas pueden tener **diferentes tamaños**.

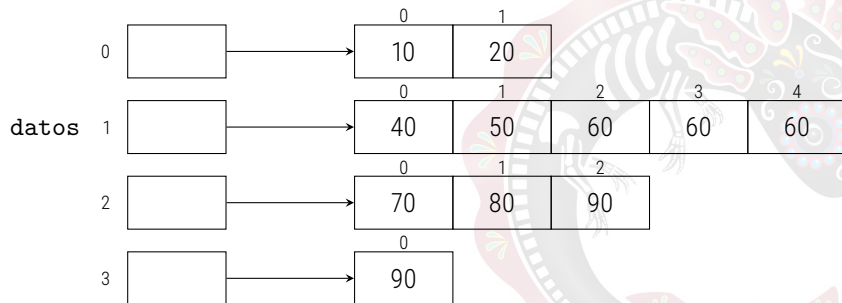
```
int[] [] datos = new int[4] [];

datos[0] = new int[2];
datos[1] = new int[5];
datos[2] = new int[3];
datos[3] = new int[1];
```

Por tanto:



Representación visual



Denominación

Estas estructuras suelen denominarse **arreglos irregulares** o *jagged arrays*.

Uso de `for-each` en matrices

Podemos utilizar ciclos `for-each` anidados para recorrer sus **filas y elementos**.

```
int [][] matriz = {  
    {10, 20, 30},  
    {40, 50, 60},  
    {70, 80, 90}  
};  
  
for (int [] fila : matriz) {  
    for (int valor : fila) {  
        System.out.print(valor + " ");  
    }  
    System.out.println();  
}
```



Importante

El **primer** `for-each` recorre las **filas**, mientras que el **segundo** recorre los **elementos de cada fila**.

Representación visual

10 20 30
fila

40 50 60
fila

70 80 90
fila

→

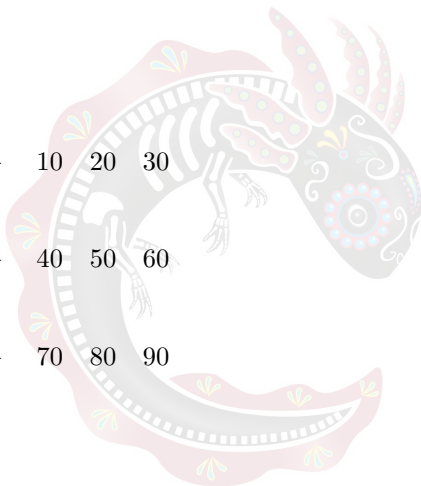
10 20 30

→

40 50 60

→

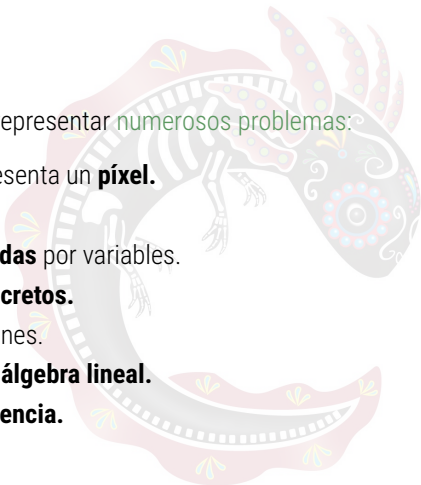
70 80 90



Aplicaciones de matrices

Los **arreglos multidimensionales** permiten representar **numerosos problemas**:

- **Imágenes digitales**: cada posición representa un **píxel**.
- **Tableros**: ajedrez, damas, sudoku, etc.
- **Datos científicos**: **mediciones organizadas** por variables.
- **Mapas**: representación de **espacios discretos**.
- **Calificaciones**: estudiantes $\times$ evaluaciones.
- **Matrices matemáticas**: operaciones de **álgebra lineal**.
- **Grafos**: matrices de **adyacencia e incidencia**.



Representación de tablas

Supongamos las calificaciones de **cuatro estudiantes** en **tres evaluaciones**:

Estudiante	Parcial 1	Parcial 2	Proyecto
Ana	8	9	10
Luis	7	8	9
María	9	10	10
Pedro	6	8	7

Los valores numéricos pueden representarse mediante:

```
double[] [] calificaciones
```

donde:

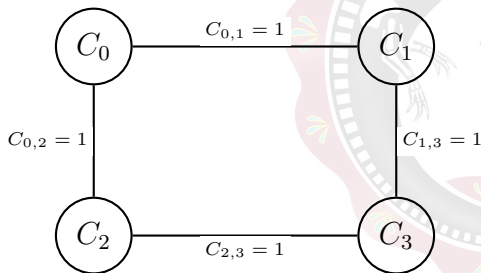
- Cada **fila** representa un **estudiante**.
- Cada **columna** representa una **evaluación**.

Representación de relaciones

Una matriz también puede representar **relaciones entre elementos**.

Ejemplo: conexiones entre computadoras

Se tiene **cuatro computadoras** conectadas de la siguiente forma:



Representación de relaciones

Podemos representar **las relaciones** de la siguiente forma:

Matriz de relaciones

Si $C_{ij} = 1$, **existe una conexión** entre los elementos i y j

si $C_{ij} = 0$, **no existe**.

$$C = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

¡Importante!

Esta representación será fundamental al trabajar con **grafos y matrices de adyacencia**.

Fin de la presentación

¿Preguntas o comentarios?

